

Dynamic Scheduling without Real-Time Requirements

Moiz Zaheer Malik
B.Eng. Electronic Engineering
Hochschule Hamm-Lippstadt
Lippstadt, Germany
moiz-zaheer.malik@stud.hshl.de

Abstract—Dynamic scheduling in hardware refers to the run-time determination of operation execution order and timing, as opposed to a fixed pre-determined (static) schedule decided at compile time. This report examines dynamic scheduling techniques in digital hardware design, specifically in the context of high-level synthesis (HLS) for systems without hard real-time constraints. We focus on hardware implementation aspects, including the formal models used to represent scheduling (such as control-dataflow graphs), the design of scheduler control logic (finite-state machines, arbitration units), and data path organization to support run-time flexibility. We review the distinctions between static and dynamic scheduling in terms of design methodology and model-based design, and discuss how dynamic scheduling is formally modeled and realized in hardware. An illustrative use case demonstrates a dynamic hardware scheduler adapting to variable execution latency. We evaluate the benefits (improved average-case performance, resource utilization, power savings) and trade-offs (hardware overhead, increased design complexity, reduced timing predictability) of dynamic scheduling versus static scheduling. The report concludes that for systems without strict real-time deadlines, dynamic scheduling can significantly enhance flexibility and efficiency in hardware, at the cost of additional architectural complexity.[1, 2, 3]

I. INTRODUCTION

High level Synthesis (HLS) Takes behavioral discription of algorithms and turns it into a hardware design. One of the important role of HLS is scheduling. Figure 1 illustrates the typical HLS flow, where after parsing and CDFG construction, the Scheduling assigns each operation in the design a specific clock cycle, It makes the decision which clockcycle and which hardware unit each operation uses. Most HLS tools produce a static schedule (every thing is pre predicted) when it's run. And in result we get fixed sequence of steps which is known as finate state machine(FSM) that controls the data flow through hardware [1]. Static Scheduling is esay to understand and like because it gives predictable timing, which is why its used in real time or safty critical systems where you need to be sure tasks finish by hard deadline [1]. But it also has drawbacks such has it wasting time, when a operation or task finishes earlier than the worst case assumption, that keeps the deadline safe but it hurts the average performance of the system and leaves the hardware sitting unused [4].

Dynamic Scheduling has a different way of scheduling insted of following a strict predetermined timeline like *Static Scheduling* it makes the decision on the fly, it reacts to actual

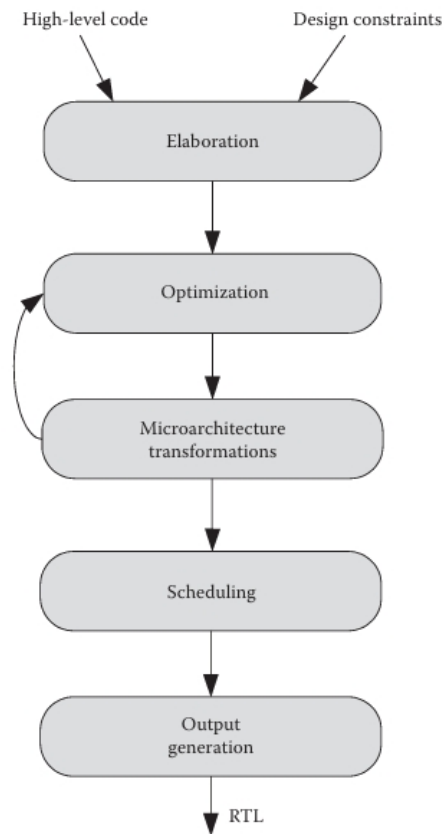


Fig. 1. Typical HLS flow [4]

data availability or operation completion events [1, 2, 3]. This is a great approach for systems where you dont need real time requirments, and the goal is to keeping the hardware busy and gettign as much as possible done is more important than worst case timing guarntees [1, 3]. By letting tasks start whenever they are ready during execution, dynamic scheduling can take advantages of chnages that hhappen at run time and allow multiple tasks to run in parallel and this is something static scheduling can not do [1, 3]. For example if there are some tasks that take more or less time depending on their data, dynamic scheduling here can handle them really well by

starting tasks as soon as they are ready, instead of waiting for the fixed time slot [3]. However this flexibility of dynamic scheduling requires extra hardware to control the scheduling, like (e.g., scheduler circuits, arbiters) which makes this makes the system less predictable, which makes testing and verifying more difficult [2], but in many modern systems such as video games and video processors this is acceptable because strict timing is not as important as overall performance [1, 4].

This report looks at how dynamic scheduling works in hardware systems, with a focus on how it's modeled and how it can be built using hardware components. In Section II, we explain the difference between static and dynamic scheduling and look at important background work and definitions. Section III introduces formal models, such as control dataflow graphs, that are used to describe scheduling problems and how dynamic scheduling fits into these models. Section IV covers how dynamic scheduling is built in hardware, including the use of control FSMs (finite state machines), arbiters for shared resources, and flexible data paths that adjust during run time. Section V gives a practical example of a dynamic scheduling method running on real hardware. In Section VI, we evaluate and compare dynamic and static scheduling, discussing the benefits, limitations, and trade offs of each approach. The report ends with a summary in Section VII.

II. BACKGROUND: STATIC VS. DYNAMIC SCHEDULING IN HARDWARE

A. Definitions and Model-Based Design

In hardware design, *static scheduling* means that the order and timing of operations are decided before the system runs during the compile time. For example, using a data flow graph or control flow graph, the HSL tool will assign each operation to a specific clock cycle and sometimes to a specific hardware unit (like an adder or multiplier) [1]. This fixed plan, known as static schedule, is built in the hardware control unit usually as a finite state machine (FSM) or a small program [1]. On the other hand, *dynamic scheduling* means the hardware decides during run time which operation to do next or when to start the next task. It makes these decisions based on what data is available, whether other operations have finished, or other conditions at that moment [2]. Dynamic scheduling is used in systems like out of order processors, dataflow based hardware, or reconfigurable systems that schedule tasks while they are running [4].

Looking at scheduling from a model based design point of view helps us understand when to use static or dynamic scheduling. Many embedded systems like DSP pipelines can be represented using synchronous dataflow graphs, where data flows steadily through blocks [1, 4]. Figure ?? shows a typical Control/Data Flow Graph (CDFG). In this graph, the nodes represent operations, and the arrows show how data or control flows from one operation to another. Because these graphs are regular and predictable, they are easy to analyze. That's why static scheduling, where all scheduling decisions are made before running, is usually the best choice. It avoids the extra cost and complexity of deciding things during runtime [1].

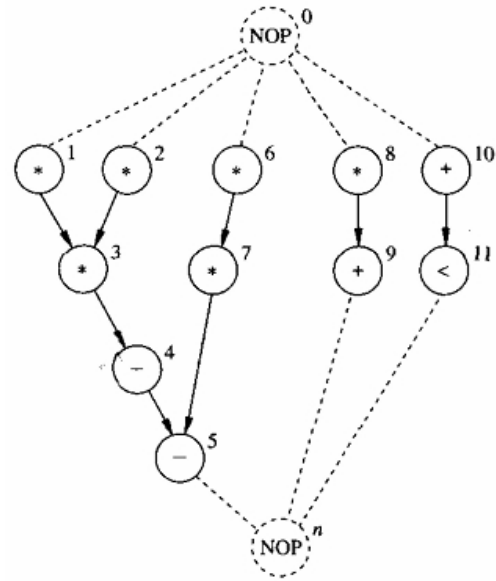


Fig. 2. Sequencing graph [2]

static scheduling is especially useful when the system behaves in a fixed and predictable way for example, when each part always produces and consumes the same amount of data. In such cases, you can create one fixed schedule ahead of time. This kind of schedule makes the system more predictable and easier to test especially when checking that time requirements are met [2, 1].

However when the system's behaviour becomes complex for example, when it includes conditions based on data, if else branches, or loops that run a different number of times then static scheduling may not work well or may not even be possible [1, 3]. In such cases, models like sequence graphs or control data flow graphs (CDFG's) are used to show how data and control signals depend on each other. But with static scheduling, you have to plan for the worst case timing, which can result in idle clock cycle while the system waits unnecessarily. When the system needs to handle asynchronous inputs or data dependent actions where the exact timing is not known ahead of time then dynamic scheduling is required [2]. It lets the hardware decide what to do next based on actual conditions while running.

Another way to compare static and dynamic scheduling is by looking at real time requirements. In hard real time systems, it's very important that all tasks finish before their deadlines. Static scheduling is a good fit here because it creates a fixed plan based on the worst case time each time each task might take [1]. This way, you can be sure the system always meets its time limits [1].

Dynamic scheduling on the other hand allows tasks to start or finish at different times depending in the situation. Sometimes a task finishes early, which improves performance, but timing can change from one run to another [2]. This makes it harder to predict and test for the worst case timing. Because of this, static scheduling is usually chosen for systems

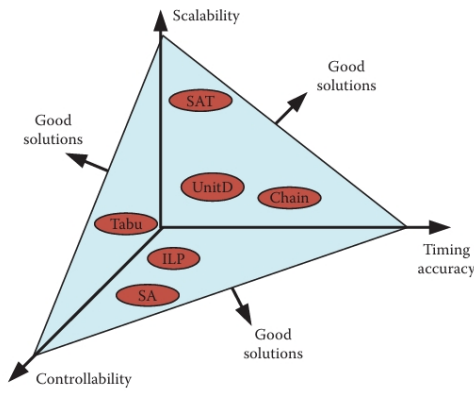


Fig. 3. Scheduling landscape [4]

that need predictable timing, even if it means the hardware isn't always used efficiently. Dynamic scheduling is preferred in systems where exact timing isn't critical, and improving average performance is more important [4, 1]

B. Static vs. Dynamic Scheduling Policies

To further illustrate the difference, consider scheduling policies ranging from fully static to dynamic [1]. Under a fully static policy, the hardware follows a fixed schedule exactly as planned during compile time. even if a task finishes early, the next task waits for its assigned time. This ensures exact timing which is good for real time system, but it can waste time when task finish faster than expected [1]. A self timed policy keeps the fixed order of tasks but allows each task to start as soon as the previous one finishes. This improves performance without needing a fully dynamic scheduler [1]. A static assignment policy gives tasks fixed hardware resources but lets the system decide at run time which ready task to run next. The order can change a little, which adds a bit of scheduling work but helps hardware better [1, 3]. in a fully dynamic policy, the system ignores the compile time schedule. it decides everything at run time based on which task are ready and which resources are free. this approach gives the best performance when workloads vary, but it needs complex control logic [3].

In summary, static scheduling is easier to control and highly predictable, but it can be less efficient because hardware might sit idle. On the other hand, dynamic scheduling uses hardware more effectively and boosts performance in system that don't need strict timing, but it adds extra control logic and makes the design more complex [1, 3]. It depends on the system which one to use. streaming DSP system usually work best with static or self timed schedules irregular or data dependent system like out of order CPUs or adaptive multimedia work better with dynamic scheduling [4].

As shown in Figure 3, fully static scheduling focuses on predictability and keeps run time control logic simple but this can lead to unused hardware on the other hand logic simple but this can lead to unused hardware. On the other end, fully dynamic scheduling offers the most flexibility and

better resource use, especially when workloads change a lot, but it comes with more complex control logic. Self timed and static assignment policies sit somewhere in between. they offer a balance providing some flexibility while keeping control overhead lower [4, 1, 3].

III. FORMAL MODELING OF DYNAMIC SCHEDULING

A. Control Dataflow Graphs and Sequencing Graphs

formal models help describe operations and how they depend on each other. One common model is the control data flowgraph (CDFG), which combines both control flow like if else decisions and data dependencies. in this graph, Nodes represent operations and Edges show the order, if one task depends on another's result, it must wait. When scheduling, we assign a start time to each operation. if an edge from operation $(u_i \rightarrow u_j)$, it means u_j must start after u_i finishes. That's written as

$$t_j \geq t_i + d_i [2]$$

Where d_i is how long operation u_i takes [2]. in static scheduling, all the t_i times are calculated ahead of time. in dynamic scheduling, the hardware figures out the start times during execution, based on what is ready and what has finished. De Micheli also describes relative scheduling, which is useful when operations take different amounts of time. in this method, the next operation waits for completion signals instead of fixed time. for example

$$t_3 = \max(t_1 + d_1, t_2 + d_2) [2]$$

Here, the system waits for both earlier tasks to finish, and the delay d_1 or d_2 can change depending on input [2]. When loops and branches (if else) are in the CDG, static scheduling becomes harder. it has to prepare for all possible paths, even ones that aren't taken. This can waste cycles. Dynamic scheduling solves this by checking conditions at run time, and only running the operations that are actually needed. this makes it more efficient, especially in complex systems [3, 1].

B. Operation Mobility and Scheduling Freedom

The concept of operation mobility comes from ASAP (As Soon As Possible) and ALAP (As Late As Possible) Scheduling. it tells us how much freedom or slack we have in deciding when an operation can start

$$\text{mobility}(u) = t_{\text{ALAP}}(u) - t_{\text{ASAP}}(u), \quad (1)$$

Here, $t_{\text{ASAP}}(u)$ is the earliest time operation u can start, and $t_{\text{ALAP}}(u)$ is the latest it can start without delaying the whole system [2]. if mobility is zero, the operation must start at a fixed time. if the mobility is greater than zero, it means there's a range of valid start times. in static scheduling, the scheduler picks one start time from that window often the ASAP time. In dynamic scheduling, the hardware can wait and decide during execution, using this slack to better manage

resources and start operations only when the data and hardware are ready [3].

Rettberg's low power high level synthesis (HLS) method uses mobility analysis to group operations into partitions. These groups can be turned on or off at run time, depending on what's needed. This saves power and adds flexibility, without breaking the system's correctness [3].

Models like petri nets or dataflow graphs work well with dynamic scheduling. In these models, operations only run or fire when their inputs are ready. This fits the idea of letting the system decide the order during execution, based on which data is available [3].

IV. HARDWARE IMPLEMENTATION OF DYNAMIC SCHEDULING

To implement dynamic scheduling, the hardware needs more than just a simple FSM controller. It also requires extra components like scheduler control logic, arbitration units to decide which task gets access to shared resources, and flexible data paths that can adjust based on which operations are active [2, 3].

A. Scheduler Control Logic and FSMs

A dynamic scheduler in hardware can be implemented using either centralized or distributed control mechanisms. In centralized control, a single controller similar to a finite state machine FSM monitors the status of operations by checking signals such as done or ready, and issues control signals at run time. This is similar to how out of order processors or scoreboard system operation in CPUs [2]. In the context of high level synthesis (HLS), this controller does not rely on a fixed schedule based on clock cycles. Instead, it reacts to events like data availability or operation completion. For example, when loops depend on run time data, the FSM includes transitions triggered by condition checks that occur during execution. More advanced scheduling policies, such as round robin or priority based issuing, can also be implemented using combinational logic and registers that evaluate which operations are ready in each cycle [2, 3].

In contrast, distributed control gives each operation block its own local control logic. These blocks communicate using handshake protocols. Each block waits for a valid input signal such as request and acknowledge and then signals its completion once the operation is done. This approach is common in self-timed or asynchronous pipelines. A notable example is Rettberg's MACT architecture, which uses self-timed local controllers that synchronize using handshake wires [3]. This method reduces the need for a single global scheduler, as decisions are made locally, though arbitration is still required when multiple blocks try to access shared resources [3].

B. Arbitration Units and Resource Control

Dynamic scheduling requires on-the-fly arbitration whenever multiple operations are ready at the same time but need the same functional unit. This is handled by arbitration units, such as priority encoders or round robin selectors, which decide

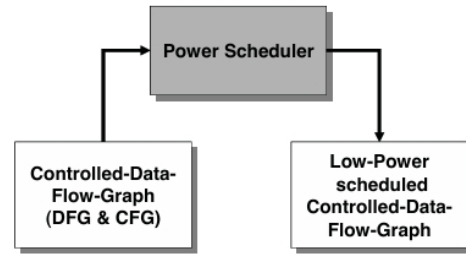


Fig. 4. Controlled Data Flow Graph [3]

which operation gets access to the shared resource. The scheduler or resource controller contains this arbitration logic to make sure there are no conflicts when several tasks want to use the same hardware at once [2, 3].

As the number of ready operations increases, the arbitration logic becomes more complex. To manage this, systems often use issue queues or buffers, where ready operations wait until a resource becomes available. These structures help ensure that operations are executed in the right order without overloading the hardware [2].

C. Data Path Considerations

Data paths in dynamic scheduling must handle changes in timing. Buffers or FIFOs are used to separate the timing when data is produced and when it is used, and valid/ready signals are used to manage these transfers [2, 3]. To allow flexible access, register files or memories may need multiple read/write ports or arbitration logic [2, 4]. Multiplexers and switches are controlled at run time to send data to the right place, ensuring that early or late results are handled correctly [1, 2].

D. Hardware Support Example

Imagine two branches running in parallel to compute X and Y , and then calculate $Z = X + Y$. One branch does multiplication that takes a variable amount of time. In dynamic hardware, the system waits for a completion signal from both branches and then triggers the adder as soon as both results are ready without assuming the worst case delay [2, 3]. This is called event-driven control, where operations start based on a done signal instead of fixed delays [2, 3]. It helps avoid idle time and improve average performance, while still handling slower operations correctly.

V. EXAMPLE USE CASE: DYNAMIC SCHEDULING IN ACTION

A practical example is the low power HLS of an MPEG-2 decoder using the MACT self-timed pipeline. ASAP/ALAP analysis was used to find operation mobility and operations were grouped into control partitions that could be dynamically turned on or off to save power and adapt to data-dependent execution paths [3]. The dynamic scheduler tracks data tokens and activates only the path the necessary parts, allowing parallel execution while powering down idle [3].

A simpler case is data dependent multiplier, it signals done when finished, and the next operation starts right away this improves average performance without needing fixed deadlines [2, 3].

After computing operation mobility on the MPEG-2 CDFG, the Power Scheduler transforms the original graph (left in Figure 4) into a low-power scheduled CDFG (right). In the transformed graph, each partition is annotated with enable/disable control based on runtime conditions, so only needed operations are activated. This dynamic CDFG is then used to generate hardware that monitors data tokens and triggers partitions as described above.

VI. EVALUATION AND DISCUSSION

Dynamic scheduling improves average performance and resource use by starting ready tasks right away and adapting to changing workloads. This helps avoid the idle cycles that static schedules often waste [4]. It's especially useful in multimedia applications with irregular control flow, where performance gains often match the amount of variable or conditional operations. Although dynamic scheduling adds some overhead, it's worth it when the workload is highly irregular. For regular tasks, static scheduling may be enough. In between, hybrid or quasi-static methods offer a good balance between flexibility and control [4].

A. Hardware Cost and Complexity

Dynamic scheduling increases the area and power needed for control logic, and makes system verification more difficult due to a larger state space and unpredictable execution order. It becomes essential to ensure correctness and avoid deadlocks in all possible scheduling scenarios. This complexity can be reduced using modular or hybrid designs, where high level scheduling remains static while local operations are scheduled dynamically [1].

B. Real Time and Predictability Considerations

In systems without hard real-time deadlines, dynamic scheduling helps improve throughput and power efficiency [1, 3]. For systems with soft real-time requirements, dynamic schedulers can still be used by incorporating static analysis or dispatch thresholds to ensure deadlines are met [1, 4]. Techniques like scheduler pipelining help maintain high clock speeds even when the control logic is complex [2].

C. When to Use Dynamic Scheduling

Dynamic scheduling is ideal for applications with irregular or data-dependent workloads, systems that do not require strict timing guarantees (such as multimedia), and designs focused on resource sharing or fine-grained power control [1, 4, 3]. In contrast, static scheduling is better for extremely high speed or safety critical systems where precise timing must be guaranteed [1]. Hybrid approaches combine the strengths of both and are well suited for many real-world designs [4, 1].

VII. CONCLUSION

Dynamic scheduling in hardware provides run-time flexibility for deciding the order of operations and allocating resources. This leads to better efficiency and power savings, especially in systems without strict timing constraints. We explored key concepts such as control-dataflow graphs (CDFGs), mobility analysis, and hardware implementations involving FSMs, arbiters, and data path buffers. A real-world use case demonstrated how dynamic scheduling can improve both performance and power efficiency. While it introduces more design complexity and makes verification harder, hybrid methods help balance flexibility and control. As hardware systems become more adaptive and diverse, dynamic scheduling will continue to play a key role alongside static methods to meet new performance and power demands.

VIII.

IX.

REFERENCES

- [1] Soonhoi Ha and Jürgen Teich, eds. *Handbook of Hardware/Software Codesign*. Provided broad context and definitions for static vs. dynamic scheduling and model-based design (data-flow models), including discussion of scheduling overhead vs. analyzability. Springer, 2017.
- [2] Giovanni De Micheli. *Synthesis and Optimization of Digital Circuits*. Provided formal models (sequencing graphs, relative scheduling), hardware control examples for dynamic scheduling, and theoretical scheduling foundations. McGraw-Hill, 1994.
- [3] Achim Rettberg. "Low Power Driven High-Level Synthesis for Dedicated Architectures". Contributed mobility-based analysis for dynamic scheduling in HLS and power-optimized hardware partitions (MACT architecture example). PhD thesis. University of Paderborn, 2006.
- [4] Louis Scheffer et al., eds. *Electronic Design Automation for IC System Design, Verification, and Testing (2nd ed.)*. Referenced for system-level design insights, scheduling in multimedia processors, and quasi-static scheduling concepts. CRC Press, 2016.